

Agile 방법론 및 MSA 개발

수업 운영 가이드

수업 진행 일정

- 1 일차 개념 실습은 오후 부터 공통으로 사전 제공된 완성형 코드 템플릿을 스스로 이해하며 정리한 **서브노트 초안을 작성**합니다.
- 1 일차 종합 실습은 **개념 실습에 대해서 실제 본인이 스스로 적용하며 겪은 시행착오나 코드 템플릿을 이해하기 위한 과정 및 경험을 서브노트에 추가**하여, 각 반별로 제출합니다.
*각 반별 제출 슬랙 스레드로 제출
- 2 일차에는, 완성된 코드 템플릿을 이해하고 활용할 수 있다는 전제로, 팀별로 기획한 아이디어에 맞춰 커스터마이징하여, 미니 AI 를 위한 MSA 서비스를 개발하는 방식입니다.
- 2 일차 실습은 다음과 같이 Agile 방법론에 따라서 2 개의 Sprint 로 진행합니다.
- → **Sprint1 워킹 스켈레톤 (walking skeleton)** 개념으로 시나리오 기획 : 시스템 전체 (**user, course, enrollment, kafka, payment, recommend**)를 동시에 조금씩 건드리는 게 아니라, 도메인 서비스를 먼저 골라서 (3 개) 그 안에서 사용자가 처음부터 끝까지 실제로 쓸 수 있는 한 가지 흐름을 완전하게 만드는 것)
- → **Sprint2 (확장) 2 단계로 진행** : 이어지는 다음 Sprint2 에서는 Sprint1 에서 손대지 않은 나머지를 다룹니다.
 - **payment-service** : 결제 도메인을 팀 아이디어에 맞게 확장
 - **recommend-service** : 추천 규칙을 팀 도메인에 맞게 확장
 - **Kafka 이벤트** : payment.completed 같은 이벤트 흐름을 붙여서, 결제 (또는 팀이 정의한 트리거) 완료 시 enrollment 상태가 자동으로 바뀌게 처리 (토픽 이름·Producer/Consumer 구조는 유지, payload 필드만 확장)
- eureka-server / auth-server / api-gateway 는 인프라이므로 실습에서 모두 수정이 대상이 아닙니다. (단, Eureka 는 서비스 등록 추가 개념을 이해할 수 있게, 소스 이해 제공)
- 인프라 2 종은 tar 파일로 직접 배포 (상세 절차는 별첨 '인프라 이미지 배포 가이드' 참고)
→ Auth Server 와 API Gateway 인프라 2 종은 이미지로 배포하고 소스 기반 실습에서는 제외합니다. SKALA 에서는 Java/ SpringBoot 에서 사용자 인증·암호화·JWT 까지는 배우지만 인프라로 제공된 셋팅은 OAuth2 (Authorization Code, Client Credentials) 확장 및 서비스 간 인증, JWK (JWT 서명 검증 공개키) 등을 포함하며, 여기까지 실습 범위에 넣으면 난이도가 갑자기 급상승하는 반면 그렇게 확대되면 SKALA 에서 목표로 하는 AI 서비스

엔지니어 과정과는 결이 달라지게 되기 때문에 수위를 조절합니다. (*이제 Java/Spring Boot 를 막 배운 상태로 가정)

상세 일정

시간	Day 1	Day 2
09:00-11:50	Agile 방법론 이론 (Scrum/Kanban, Sprint-Backlog-Standup-Review-Retro, User Story, DoD)	Day1 회고 리마인드 + Sprint2 Planning
	시나리오/아키텍처 설명 + 완성 코드 템플릿 데모 (docker compose up, Eureka 대시보드, 로그인→강의조회→수강신청→결제→추천 전체 흐름)	Sprint1 실행 (오전), Sprint2 착수
12:00-13:00	점심	점심
13:00-14:50	완성 코드 템플릿 설명 (docker compose up, Eureka 대시보드, 로그인→강의조회→수강신청→결제→추천 전체 흐름)	Sprint2 실행 (오후)
	팀 편성 + 아이디어 기획 킥오프	
16:00-18:00	Sprint1 Planning (팀별 시나리오 확정, 담당 역할/디렉토리 배분) Sprint1 실행 Sprint1 Review (데모) + Retro	통합 테스트 및 버그픽스 (버퍼) 최종 데모(팀별 발표) + 전체 회고

실습 서브 노트 (1 일차)

- 1. 이해관계자 가치 (Pain Point) :** AI 서비스 개발에서 MVP(Minimum Viable Product)가 어떤 경우에 적용되며, 이 과정에서 Agile과 MSA(Microservice Architecture) 적용이 왜 필요하고 이 기술들이 어떤 이해관계자에게 어떤 가치를 제공할 수 있을 지 제시합니다. Agile과 MSA 방식 접근이 없다면, AI 서비스 이해관계자가 실제로 겪는 불편함이 무엇인지, 왜 그 문제가 중요한지를 먼저 명확히 정의합니다.
- 2. 이를 해결하기 위한 AI 솔루션 :** 정의한 Pain Point를 어떻게 해결할지, 이해관계자 가치를 제시할 수 있도록 AI 업무 서비스들을 기획합니다. 여기에서 관련된 핵심 기능과 그 안에서

Agile과 MSA가 담당하는 역할을 설명합니다.

3. 아키텍처 구성도 : 이해관계자가 "이 솔루션이 기술적으로 어떻게 동작하는가?"를 구조적으로 이해할 수 있게 하는 것이 목적입니다. (경영진, 업무 담당자, 기술 책임자 등)

실습에서는 MSA로 개발된 완성된 백엔드 REST API를 제공합니다. 추가로 규칙기반 추천 모델 포함을 포함하고 있습니다. AI 서비스 측면에서 Agile & MSA 기반 가치를 줄 수 있는 특성을 시나리오(User Story)로 도출하고 이를 적용해 Agile기반으로 MSA 특성을 활용하여 AI 서비스가 만들어지기까지의 전체 파이프라인을 한눈에 보이도록 구성합니다.

특히 다음 질문에 대한 답이 제시되어야 합니다. **1) 실습에서 요구사항(Backlog) 우선 순위를 어떻게 도출할** 것이며, 이에 따라서 실습에서는 Sprint 1 및 Sprint2를 어떤 기준으로 나눌 것인가?

2) MSA 분할 설계 원칙에 대한 이해

- Microservice는 업무 단위로 분할하는데, 통상 업무 결과가 데이터로 생성되기 때문에 데이터 생성/처리 그룹 단위로 분할하는 것이 일반적이다. 제시된 코드 템플릿은 Microservice 들이 어떤 기준으로 나뉘어져 있는가?

- 같은 데이터를 다루는 트랜잭션들은 MSA로 분리할 수 없다. 그럼 하나의 서비스 안에서 이 각각의 트랜잭션들은 어떻게 격리화해야 하는가?

- 분할된 MSA들은 식별/인증/통신 대상이 된다. 이를 위해서 필요한 인프라 특성을 설명할 수 있는가? 참고로 이 인프라 특성은 이번 실습 대상이 아니며 완성된 형태로 제공됩니다. (실무에서도 신입 개발자에게 인프라 구축까지 맡기진 않음)

4. 트러블 슈팅 : 실습 과정에서 경험하고 정리한 내용을 기록합니다. 새로운 용어에 대한 이해일 수도 있고, 요소기술간의 연결일 수도 있습니다. 또는 트러블 슈팅 경험을 기록합니다.

5. 레퍼런스 (필요한 스냅샷) : 교수자 강의 내용을 키워드 중심으로 요약하고 정리해 두었다가, 서브 노트 작성 시 활용합니다. 수업에서 교수자가 설명하고 본인이 이해한 내용이 실제로 구현 · 검증되었다는 증빙 자료 역할을 합니다.

Sprint1 / Sprint2 기준

Sprint1은 팀이 기획한 새 도메인을 course-service-enrollment-service 등 서비스 2~3개 안에서만 좁게 적용해, 팀 아이디어에 맞게 로그인부터 신청까지 한 시나리오가 처음부터 끝까지 동작하게 만드는 것이 목표입니다. payment-kafka-recommend는 원본 그대로 두고 통과시킵니다.

Sprint2는 Sprint1에서 손대지 않은 나머지 서비스(결제, Kafka 이벤트, 추천)까지 팀 아이디어에 맞게

확장해 전체 플로우를 완성하는 것이 목표입니다.

인프라 배포 방식 (요약)

담당 전임 교수가 auth-server-api-gateway 2개 이미지를 빌드해 infra-images.tar로 저장하고, 슬랙을 통해 실습 자료와 함께 제공합니다. 배포 패키지 구성 : infra-images.tar + 수정된 docker-compose.yml + 학생이 실제로 수정하는 서비스 소스(course-service, enrollment-service, payment-service, recommend-service, vue-frontend, init-db). 단, 예고 드렸듯이 auth-server 및 api-gateway 디렉토리는 인프라 성격으로써 수업 난이도가 높아져서 이번에는 제외합니다. 실무에서도 신입 개발자가 이런 인프라 구축까지 진행하지 않습니다.

이해도를 높이기 위한, Day1 설명 보완 사항

학생들은 Java-Spring Boot 기초와 사용자 인증·암호화·JWT까지는 학습했지만, OAuth2·Eureka·Kafka·Gateway·FastAPI·Vue 및 프론트엔드~백엔드 연동은 이번에 처음 접합니다. 이에 따라 Day1 이론 시간에 "지금까지 배운 JWT 인증에서, 서비스가 여러 개로 쪼개지면 무엇이 새로 필요해지는가?" (토큰 발급자와 검증자의 분리, JWK, 헤더 전파)를 짧게 연결하는 개념 설명을 포함합니다.

이를 통해 학생들이 코드를 직접 수정하지 않아도 auth-server/gateway 코드 리뷰를 이해할 수 있도록 제공합니다.

각 반별 체크리스트

- ☐ 각 반에서 개인별 infra-images.tar + 수정된 docker-compose.yml + 학생용 소스 (course/enrollment/payment/recommend/vue-frontend/init-db) 전달 확인
- ☐ 각 반에서 개인별 docker load + docker compose up -d 동작 테스트 점검
- ☐ eureka-server / auth-server / api-gateway 는 학생이 수정하지 않는다는 점 실습 시작 시 공지
- ☐ Sprint1 : 팀별 기획 시나리오 1 개 확정 및 담당 서비스 역할/디렉토리 배분 확인
- ☐ Sprint1 종료 시 팀별 짧은 데모 + 미니 회고 진행
- ☐ Sprint2 : payment/kafka/recommend 확장 범위 확정, 필요 시 신규 서비스+게이트웨이 라우팅 계획 포함
- ☐ Day2 최종 데모 및 전체 회고 진행

실습 배포 패키지 (전달 항목)

항목	내용
이미지 파일	infra-images.tar (auth-server, api-gateway) msa-lecture-images.part.aa, msa-lecture-images.part.ab, msa-lecture-images.part.ac (Microservice)
docker-compose.yml	각 서비스 도메인을 컨테이너 이미지로 빌드 배포하는 구성 정보
소스 디렉토리	course-service, enrollment-service, payment-service, recommend-service, vue-frontend, init-db (학생이 실제로 수정하는 것만)
제외 항목	auth-server/, api-gateway/ 인프라성 디렉토리는 Microservice 구현에 필요한 셋팅으로써 별도 이미지로 제공 (수업 목적 및 난이도에 맞지 않아 제외)

실습 시, 개인 PC 공통 실행 절차

실습에서는 Microservice로 구현된 완성형 코드 템플릿을 제공합니다.

REST API로 구현된 Microservice를 모두가 동시 빌드하게 되면 대량 네트워크 트래픽이 사용되므로, 실습에서는 신규 빌드하여 사용하지 않고, 사전에 빌드된 이미지로 사용합니다.

첫번째 배포 파일들 압축을 풀고 msa-lecture 프로젝트 폴더로 이동합니다.

→ msa-lecture.zip 이 파일은 MSA로 구현된 SpringBoot 및 FastAPI 소스 코드와, 공통 인프라 이미지를 포함하고 있습니다.

1. 기존 **docker-compose.yml** 백업합니다. 기존 docker-compose.yml 파일은 REST API 신규 빌드용입니다.

```
mv docker-compose.yml docker-compose.build.yml
```

2. 배포받은 로컬 이미지용 파일을 **docker-compose.yml**로 변경합니다.

```
mv docker-compose.local.yml docker-compose.yml
```

같이 배포된 추가 이미지 파일들도 다운 받아 msa-lecture 프로젝트 폴더로 이동합니다.

→ msa-lecture-images.part.aa, msa-lecture-images.part.ab, msa-lecture-images.part.ac 이 파일들은 MSA 구현된 SpringBoot 소스 코드가 빌드된 이미지 파일입니다.

3. 지금 첨부한 파일들을 해당 프로젝트 폴더에 놓고 합칩니다.

```
cat msa-lecture-images.part* > msa-lecture-images.tar.gz
```

4. 서비스 이미지 파일을 로딩합니다. (Microservice)

```
docker load -i msa-lecture-images.tar.gz
```

5. 공통 인프라 이미지 파일을 로딩합니다. (Auth Server, API Gateway, Kafka)

```
docker load -i infra-images.tar
```

6. 모든 이미지들이 정상적으로 등록되었는지 확인합니다. msa-lecture/auth-server:1.0 등 태그

```
docker images ls
```

7. 이미지를 신규 빌드하지 않고 서비스를 실행합니다.

```
docker compose up -d --no-build --pull never
```

주의 : 참고로 다음 명령은 "이미지 빌드 및 컨테이너 생성" 명령입니다. 이번에는 절대 실행하지 않습니다.

향후 개인 실습 시 활용해 보시기 바랍니다.

```
docker compose build --no-cache
```

```
docker compose up -d
```

또는 한줄로

```
docker compose build --no-cache && docker compose up -d
```

이후 course-service 등 학생이 수정한 서비스만 `docker compose up -d --build <서비스명>`으로 재 빌드하면 됩니다.

제공되는 Microservice 에 대한 REST API 계약 정보 확인

<http://localhost:8081/swagger-ui/index.html> > user-service : SpringBoot

<http://localhost:8082/swagger-ui/index.html> > course-service : SpringBoot

<http://localhost:8083/swagger-ui/index.html> > enrollment-service : SpringBoot

<http://localhost:8084/swagger-ui/index.html> > payment-service : SpringBoot

<http://localhost:8085/docs> > recommend-service : FastAPI

이번 프로젝트에서는 MariaDB 컨테이너가 3379 포트를 사용합니다.

다른 수업에서 3379 포트를 사용하는 MariaDB 컨테이너를 이미 실행 중이라면, 포트 충돌이 발생할

수 있으므로 기존 컨테이너를 중지한 후 프로젝트를 실행해 주시기 바랍니다.

Agile & MSA 실습 가이드

무엇을 이해하고, 무엇을 몰라도 되는가?

1. 왜 Agile 과 MSA 를 함께 배우는가?

소프트웨어 개발에서 가장 어려운 문제 중 하나는 "요구사항이 개발 도중에 계속 바뀐다"는 것입니다. 예전에는 처음에 완벽한 설계도를 그려놓고 그 순서대로 개발하는 방식(폭포수 방식)을 많이 사용했지만, 이 방식은 중간에 요구사항이 바뀌면 이미 만든 부분을 통째로 다시 만들어야 하는 문제가 있었습니다. 이런 경험을 직접 해본 적이 없더라도, AI 서비스처럼 빠르게 변화하는 분야에서는 처음에 세운 계획이 끝까지 그대로 유지되는 경우가 오히려 드물다는 점을 떠올려보면 쉽게 이해될 것입니다.

Agile 방법론은 이 문제를 "한 번에 완벽하게 만들기"가 아니라 "작게 나눠서 자주 만들고, 피드백을 반영해 계속 고쳐나가기"로 해결합니다. 이번 실습에서 스프린트1과 스프린트2로 나눈 것이 바로 이 방식입니다.

그런데 Agile 방식으로 "자주 고치고 다시 배포"하려면, 코드 구조 자체도 그것이 가능하도록 짜여 있어야 합니다. 만약 모든 기능이 하나의 거대한 프로그램으로 얹혀 있다면, 결제 기능 하나만 고쳐도 회원 기능·과목 기능까지 전부 다시 배포해야 하고 오류 위험도 커집니다. MSA(마이크로서비스 아키텍처)는 기능을 user-service, course-service, payment-service처럼 독립된 서비스 단위로 쪼개어, 한 서비스만 고쳐서 배포해도 다른 서비스에 영향을 주지 않도록 만드는 구조입니다. 즉 MSA는 Agile이 요구하는 "빠르고 잦은 변경"을 기술적으로 뒷받침해주는 아키텍처입니다.

이번 실습에서 여러분에게 요구되는 능력은 화면을 예쁘게 만드는 것이 아닙니다. 기획서를 쓰듯이 "이 AI 서비스가 어떤 이해관계자(사용자, 강사·판매자, 운영자 등)에게 어떤 가치를 주는가?"를 먼저 정의하고, 그 가치를 이번에 배우는 기술과 연결할 수 있는 능력이 핵심입니다. 그 가치를 어떤 기능은 스프린트1에 넣고 어떤 기능은 스프린트2로 미룰지 우선순위로 나누는 것, 그리고 각 서비스가 어떤 API를 주고받아야 그 가치가 실제로 전달되는지 명세를 설계하는 것 자체가 이번 실습에서 가장 중요하게 다뤄야 할 기획 역량입니다.

또 한 가지 기억할 점은, 이번 실습이 다루는 것은 전통적인 소프트웨어 개발이 아니라 AI 특성을 가진 소프트웨어 개발이라는 것입니다. 전통적인 소프트웨어는 요구사항이 비교적 명확하고 안정적인 경우가 많지만, AI 서비스는 모델 성능·사용자 반응·데이터에 따라 결과와 요구사항이 계속 바뀌고, 새로운 모델이나 에이전트 도구가 끊임없이 실험되고 교체됩니다. 이런 불확실성과 잦은 변화가 AI 소프트웨어 개발의 본질적인 특성이기 때문에, 작게 나눠 자주 검증하는 Agile과 모듈 단위로 독립적으로 교체 가능

한 MSA가 유난히 잘 맞는 조합입니다. 여러분이 AI 서비스 엔지니어 커리큘럼에서 이 수업을 듣는 이유도 바로 여기에 있습니다.

정리하면, Agile은 "어떻게 일할 것인가"에 대한 방법론이고, MSA는 그 일하는 방식을 가능하게 하는 "기술 구조"입니다. 이 둘은 AI 소프트웨어 개발의 불확실성과 특히 잘 맞기 때문에 세트로 함께 쓰이며, 이번 실습은 이해관계자 가치 정의 → 스프린트 구분 → API 명세 설계로 이어지는, 앞으로 여러분이 AI 서비스 에이전트를 개발할 때 실제로 마주하게 될 기획·개발 방식을 미리 경험해보는 과정입니다.

2. 실무에서는 이렇게 활용된다 (Agile·MSA의 진화 흐름)

많은 기업이 처음부터 MSA로 시작하지 않습니다. 스타트업이나 신규 서비스는 보통 하나의 프로그램 안에 모든 기능을 넣은 모놀리식(Monolith) 구조로 빠르게 만들어 시장에 먼저 선보입니다. 초기에는 팀 규모도 작고 기능도 단순해서, 오히려 하나로 뭉쳐 있는 편이 개발 속도가 빠르기 때문입니다.

그런데 서비스가 성장해 사용자와 기능이 늘어나면 문제가 생깁니다. 회원·결제·추천 기능이 하나의 코드에 얹혀 있으면 결제 로직 하나를 고치기 위해 전체 프로그램을 다시 테스트하고 배포해야 하고, 팀원이 늘어날수록 같은 코드베이스를 서로 건드리다 충돌도 잦아집니다. 이 시점에서 많은 회사가 기능 단위로 서비스를 쪼개는 MSA로 전환합니다. 특히 개인 사용자별로 특화된 서비스를 제공하는 엔터프라이즈 유형 중에서 Netflix, Amazon, 배달의 민족처럼 트래픽과 조직 규모가 큰 서비스일수록 MSA를 채택하는 이유가 여기에 있습니다.

MSA로 전환하면 배포와 확장은 자유로워지지만, 그 대가로 서비스 간 통신 관리, 데이터 일관성 유지, 운영 복잡도 증가라는 새로운 부담이 생깁니다. 그래서 실무에서는 "무조건 잘게 쪼갬다"가 아니라, 조직 규모와 서비스의 변경 빈도를 보고 얼마나 나눌지를 판단합니다. 최근에는 지나치게 잘게 쪼갬 MSA의 운영 부담을 줄이기 위해 관련 기능을 조금 더 큰 단위로 묶는 모듈러 모놀리식(Modular Monolith) 같은 절충안도 함께 논의되고 있습니다.

여러분이 이번 실습에서 다루는 것은 이미 MSA로 나뉜 상태의 서비스입니다. 하지만 실무에서는 이 구조에 도달하기까지 "왜 나눴는지", "얼마나 나눌지"를 판단하는 과정이 있었다는 것을 기억해 두면, 앞으로 실제 프로젝트에서 아키텍처를 설계할 때 훨씬 균형 잡힌 시각을 가질 수 있습니다.

3. 시작하기 전에 꼭 알아야 할 것

이번 실습의 목표는 MSA 인프라(Eureka, API Gateway, 인증 서버, Kafka)를 처음부터 설계하고 구현하는 능력이 아닙니다. 이미 만들어진 MSA 구조를 실무에서 신입 개발자가 그러하듯 "가져다 쓰는" 능력을 기르는 것이 목표입니다.

실제 회사에서도 신입 개발자가 입사 첫 주에 인프라팀이 구축한 인증 서버나 서비스 디스커버리 설정을 한 줄씩 다 이해하고 시작하지는 않습니다. 정해진 API 명세를 보고 그에 맞춰 화면을 만드는 것이 실제 협업 방식입니다.

제공된 백엔드 코드를 한 줄씩 읽으며 이해하려는 시도는 이번 실습 범위를 벗어난 것이며, 이해가 안 되는 것이 정상입니다.

4. 이해해야 할 것 vs 몰라도 되는 것

구성요소	알아야 할 것	몰라도 되는 것
user-service	회원가입·로그인 요청 형식, 응답으로 오는 토큰	비밀번호 암호화 로직, DB 내부 구조
course-service	상품(과목) 등록·조회 API 형식	JPA 매핑, 내부 쿼리
enrollment-service	"신청하면 이벤트가 발생하고, 결제 후 자동으로 상태가 바뀐다"는 흐름	Kafka Producer/Consumer 코드
payment-service	결제 완료 호출 → 이후 enrollment 상태가 바뀐다는 결과	결제 처리 내부 로직
auth-server / api-gateway / eureka-server	로그인하면 토큰이 나오고, 그 토큰을 헤더에 넣으면 다른 API가 동작한다는 사용법	내부 설정, 서명 알고리즘, 서비스 등록 방식

이 표가 핵심입니다. "몰라도 되는 것" 칸에 있는 내용을 이해하려고 애쓰다가 막힌 것이라면, 그건 여러분의 문제가 아니라 실습 범위를 벗어난 질문을 스스로에게 던진 것입니다.

5. 본인 아이디어로 치환하는 법 (도메인 매핑표)

템플릿은 온라인 교육 플랫폼(강사•과목•수강신청•결제)이지만, 개념 구조만 같으면 어떤 서비스로도 치환할 수 있습니다. 다음 예시는 B2B 또는 B2BC 형태로 이해 바랍니다. 참고로 우리는 기업에게 가치를 제공하는 것이고, 해당 기업이 B2C를 하는 것입니다. 개발자가 직접 B2C 업무를 해야 하는 경우는 제외 바랍니다.

템플릿 개념	우리 팀 아이디어 (예 : 쇼핑몰이라면)
강사	판매자
과목 등록	상품 등록
수강신청	장바구니 담기 / 주문
결제	결제
수강권한 노출	주문 확정 후 상품 접근 허용
(빈칸)	직접 채워보세요 →

팀별로 이 표를 먼저 채우고 시작하세요. 표가 채워지면, 여러분이 만들 화면과 호출할 API가 거의 자동으로 정해집니다.

6. 이 프로젝트 구조에서 Agile•MSA 가 실제로 어떻게 드러나는가

1번에서 설명한 원리가 이번 실습에서는 구체적으로 이렇게 구현되어 있습니다. 스프린트1(회원•과목•수강신청)에서 먼저 핵심 기능만 동작하는 최소 버전(MVP : Minimum Viable Product)을 완성하고, 스프린트2(결제•이벤트 연동)에서 기능을 점진적으로 확장합니다. 이때 결제 기능을 새로 추가해도 스프린트1에서 만든 회원•과목 서비스는 건드릴 필요가 없는데, 이는 각 기능이 독립된 서비스(MSA)로 분리되어 있기 때문에 가능한 일입니다.

Agile의 "점진적 확장"과 MSA의 "독립된 서비스 단위"가 서로 맞물려야만 스프린트를 이렇게 자연스럽게 나눌 수 있습니다. 여러분이 지금 겪고 있는 실습 구조 자체가 그 실제 사례입니다.

스프린트(스크럼)를 나누는 기준

이사할 때를 떠올려보면 쉽습니다. 이사 당일 짐을 한 번에 다 옮기지 않고, 없으면 당장 생활이 안 되는 필수품(침대, 냉장고)부터 먼저 옮기고, 있으면 편하지만 없어도 며칠은 버틸 수 있는 물건(장식품, 여분 가구)은 나중에 옮깁니다. 스프린트를 나누는 기준도 동일합니다. "이 기능이 없으면 사용자에게 서비스

로서 가치를 줄 수 없는가?"를 기준으로, 없으면 안 되는 핵심 기능(회원가입·상품·신청)은 스프린트1로, 있으면 더 좋지만 없어도 서비스 자체는 굴러가는 기능(결제 자동화, 이벤트 연동)은 스프린트2로 미룹니다.

스크럼을 통해 실제로 얻는 것

스프린트를 나누는 이유는 단순히 "일정을 쪼개기 위해서"가 아닙니다. 첫째, 스프린트1이 끝나면 완성을 기다리지 않고 바로 동작하는 결과물로 피드백을 받을 수 있습니다. 둘째, 설계가 잘못된 부분(예: 회원 인증 흐름 자체의 문제)을 프로젝트 초반에 발견할 수 있어, 다 만든 뒤에야 문제를 발견하는 폭포수 방식의 위험을 줄여줍니다. 셋째, 스프린트1의 피드백을 보고 스프린트2의 계획을 바꿀 수 있는 유연성이 생깁니다. 예를 들어 스프린트1을 시연해보니 결제보다 검색 기능이 더 급하다는 것을 알게 되면, 스프린트2의 우선순위를 그때 조정하면 됩니다.

팀원 역할에 따라 개인 과제가 달라져도 되는가?

네, 오히려 그래야 정상입니다. 같은 요리를 만드는 팀이라도 한 사람은 재료를 준비하고, 한 사람은 조리를 하고, 한 사람은 플레이팅을 맡는 것처럼, 스크럼 팀 안에서도 기획(Pain Point·우선순위 정의), 백엔드(API 구현·명세 작성), 프론트엔드(화면 구현·API 연동) 등 역할에 따라 개인 과제의 내용은 달라지는 것이 맞습니다. 중요한 것은 각자의 결과물이 "같은 스프린트 목표"로 다시 모여야 한다는 점입니다. 서로 다른 산출물이 API 명세라는 공통 약속을 통해 연결되기 때문에, 백엔드 담당이 서비스를 만드는 동안 프론트엔드 담당은 Swagger 문서만 보고 화면을 미리 만들 수 있습니다. 이것이 바로 MSA가 팀 협업 측면에서도 유리한 이유입니다: 서비스가 독립적으로 나뉘어 있기 때문에 팀원들도 서로를 기다리지 않고 병렬로 작업할 수 있습니다.

7. 조별 발표 기획서에 포함해야 할 내용

발표 기획서는 아래 여섯 가지 흐름으로 구성해 주세요. 각 항목은 앞 단계의 결과가 다음 단계의 근거가 되도록 순서대로 이어져야 합니다.

- 이해관계자 가치(Pain Point)**: 우리 팀이 정의한 이해관계자(사용자, 강사·판매자, 운영자 등)가 실제로 겪는 불편함이 무엇인지, 왜 그 문제가 중요한지를 먼저 명확히 정의합니다. (5번 도메인 매핑표에서 정한 이해관계자를 기준으로 작성)
- 이를 해결하기 위한 AI 솔루션**: 정의한 Pain Point를 어떻게 해결할지, 우리 서비스가 제공하는 핵심 기능과 그 안에서 AI가 담당하는 역할을 설명합니다.
- 스프린트 구분**: 정의한 솔루션 중 어떤 기능을 스프린트1(핵심 가치, MVP)에서 먼저 구현하고 어떤 기능을 스프린트2(확장 기능)로 미루지 우선순위를 정하고, 그렇게 나눈 이유를 설명합니다. (6번에서 다룬 스프린트1·스프린트2 구조를 우리 팀 서비스에 맞춰 적용)

4. **아키텍처 구성도** : 제공된 MSA 구조(user-service, course-service, enrollment-service, payment-service, Eureka, API Gateway, 인증 서버, Kafka)를 우리 팀 서비스명으로 치환한 다이어그램입니다. 어떤 서비스가 무엇을 담당하는지, 서비스 간 호출·이벤트 흐름을 화살표로 표시합니다.
5. **API 명세** : 프론트엔드가 실제로 호출하는 엔드포인트 목록(Method, URL, Request/Response 예시)입니다. Swagger UI에서 확인한 내용을 정리하면 됩니다. (Swagger UI를 보고 실제로 프론트엔드와 연결하는 방법은 부록 참고)
6. **동작 화면 스냅샷** : 실제로 만든 화면에서 위 API가 호출되어 동작하는 장면을 캡처합니다. 요청 전/후 상태 변화가 드러나도록 구성해 주세요 (예 : 수강신청 전 → 결제 → 신청 후 화면 노출).

이 여섯 단계는 실무에서 쓰이는 서비스 기획서의 기본 구조와 동일합니다. "왜(Pain Point) → 무엇을(솔루션) → 어떻게 나눠서(스프린트) → 어떻게 구현(아키텍처·API) → 결과(화면)"의 순서를 지키면, 발표를 듣는 사람도 우리 팀의 논리를 자연스럽게 따라올 수 있습니다.

8. 실습 진행 순서 (체크리스트)

1. 팀 아이디어 확정 → 위 도메인 매핑표 작성
2. 제공된 API 문서를 보고 필요한 엔드포인트만 리스트업 (전체 코드를 읽을 필요 없음)
3. Swagger UI에서 해당 엔드포인트를 직접 호출(Try it out)해보고 요청/응답 형식만 확인 (백엔드 코드는 보지 않아도 됨)
4. HTML/JS(fetch)로 화면을 만들고, 3번에서 확인한 엔드포인트를 연결
5. 로그인 후 받은 토큰을 저장해두고, 이후 모든 요청 헤더에 포함
6. 스프린트2 : 결제 API를 호출한 뒤, enrollment 상태가 바뀌는지 "결과"만 확인 (Kafka 내부 동작은 몰라도 됨)

9. 막혔을 때 질문하는 법

질문을 던질 때 "Eureka가 왜 이렇게 짜여 있나요?"처럼 인프라 내부를 묻지 마시고, "이 API에 이 요청을 보냈는데 왜 이런 응답이 오나요?"처럼 본인이 실제로 호출한 지점으로 좁혀서 질문해 주세요.

질문 범위를 좁히면 훨씬 빠르게 답을 얻을 수 있고, 야간 보강 시간에도 이 레벨의 질문을 우선적으로 다루겠습니다.

부록 1. Swagger UI 로 프론트엔드·백엔드 연결하기

일부 팀은 백엔드 연결 방법을 몰라 목업(mock) 데이터만으로 화면을 만들고 실제 연결은 건너뛰었습니다. 이 부록은 Swagger UI만 보고도 프론트엔드에서 실제 API를 호출할 수 있도록 단계별로 안내합니다. 여기까지만 따라오면, 백엔드 코드를 몰라도 실제로 동작하는 화면을 만들 수 있습니다.

1단계. Swagger UI 구조 읽기

API Gateway 주소 뒤에 Swagger 문서 경로로 접속하면, 서비스별로 태그가 묶여 있고 그 아래 Method(GET/POST/PUT/DELETE)와 경로(Path)가 나열되어 있습니다. 각 항목을 펼치면 Parameters(요청에 필요한 값), Request body(POST/PUT일 때 보낼 JSON 형식), Responses(응답 형식과 예시)를 볼 수 있습니다. 이 세 가지가 프론트엔드 코드를 작성하는 데 필요한 전부입니다.

2단계. Try it out으로 먼저 직접 호출해보기

코드를 짜기 전에 Swagger UI의 "Try it out" 버튼을 눌러 값을 입력하고 Execute를 눌러보세요. 어떤 요청을 보내야 어떤 응답이 오는지 코드 없이 먼저 눈으로 확인할 수 있습니다. Execute 후 화면 하단에 나오는 curl 명령어를 복사해두면, 이후 코드로 옮길 때 그대로 참고할 수 있습니다.

3단계. 인증이 필요한 API는 토큰부터 발급

로그인 API를 Try it out으로 먼저 호출해 응답으로 오는 토큰을 복사합니다. Swagger UI 우측 상단의 "Authorize" 버튼을 누르고 "Bearer 복사한토큰" 형식으로 입력하면, 이후 모든 요청에 인증 헤더가 자동으로 포함되어 인증이 필요한 API도 바로 테스트할 수 있습니다.

4단계. 확인한 내용을 그대로 프론트엔드 코드로 옮기기

Swagger에서 확인한 Method·경로·Request body를 fetch로 옮기면 되는데, 이 fetch 코드 자체는 "어딘가에 떠 있는" 것이 아니라 반드시 HTML의 <script> 태그 안에, 그것도 "버튼을 눌렀을 때" 실행되도록 이벤트 리스너 안에 넣어야 합니다. 로그인 화면 전체를 예로 들면 다음과 같습니다.

```
<!-- login.html -->
<input id="email" type="text" placeholder="이메일">
<input id="password" type="password" placeholder="비밀번호">
<button id="loginBtn">로그인</button>

<script>
  document.getElementById("loginBtn").addEventListener("click", () => {
    fetch("http://localhost:8080/api/users/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        email: document.getElementById("email").value,
```

```

        password: document.getElementById("password").value
    })
})
.then(res => res.json())
.then(data => {
    sessionStorage.setItem("token", data.token); // 다음 화면에서도 쓰기 위해 저장
    location.href = "course-list.html"; // 로그인 성공 후 다음 화면으로 이동
});
});
</script>

```

핵심은 두 가지입니다. 첫째, fetch는 addEventListener("click", ...) 안에 있어야 "버튼을 눌렀을 때" 동작합니다. 둘째, 로그인에서 받은 토큰은 변수(const token)에만 담아두면 페이지를 이동하는 순간 사라지므로, 여러 화면에서 계속 쓰려면 sessionStorage 같은 브라우저 저장소에 저장해야 합니다.

다음 화면(course-list.html)에서는 페이지가 열리자마자 저장해둔 토큰을 꺼내 인증이 필요한 API를 호출합니다.

```

<!-- course-list.html -->
<ul id="courseList"></ul>

<script>
    const token = sessionStorage.getItem("token");

    fetch("http://localhost:8080/api/courses", {
        method: "GET",
        headers: { "Authorization": "Bearer " + token }
    })
    .then(res => res.json())
    .then(courses => {
        const list = document.getElementById("courseList");
        courses.forEach(c => {
            const li = document.createElement('li');
            li.textContent = c.title;
            list.appendChild(li);
        });
    });
</script>

```

즉 "로그인 화면의 스크립트"와 "과목 목록 화면의 스크립트"는 서로 다른 HTML 파일에 각각 들어가고, 그 사이를 sessionStorage에 저장한 토큰이 이어주는 구조입니다. 화면(HTML)마다 필요한 API를 이런 식으로 하나씩 연결해 나가면 됩니다.

5단계. 자주 겪는 오류 : CORS

프론트엔드에서 호출했는데 브라우저 콘솔에 CORS 오류가 뜬다면, 개별 서비스 포트(8081~8085)로 직접 호출했기 때문일 가능성이 큼니다. 반드시 API Gateway 주소(8080)로 호출해야 합니다. 호출 주소가 http://localhost:8080/... 형태인지부터 확인하세요.

6단계. 그래도 막히면

목업으로 넘어가기 전에 1~3 단계(Swagger에서 Try it out으로 성공하는 요청 확인)까지는 반드시 해보

세요. Swagger에서 이미 성공한 요청이라면, 프론트엔드 코드로 옮기는 것은 그 URL·헤더·body를 그대로 옮기는 것뿐입니다. 이 단계에서 막혔다면 9번 항목의 방법대로 구체적으로 질문해 주세요.

부록 2. 자주 묻는 질문 (Q&A)

지금까지 나온 질문과 답변을 모아두었습니다. 본문에서 이미 다룬 내용도 있지만, 헛갈렸던 부분만 빠르게 다시 찾아볼 수 있도록 여기에 정리했습니다.

Q. fetch로 REST API를 연결하는 게 맞나요? JSON 형식으로 주고받으니 REST API로 봐도 되나요?

A. 네, 맞습니다. REST API는 URL(자원)·HTTP Method(GET/POST/PUT/DELETE)·JSON 등으로 데이터를 주고받는 "설계 방식"이고, fetch는 그 REST API를 브라우저에서 호출하는 "도구"일 뿐입니다. 지금 코드처럼 URL로 자원(users, courses)을 지정하고 Method로 동작을 구분하고 JSON으로 데이터를 주고받고 있다면 REST API 방식이 맞습니다. fetch 대신 axios나 Postman·curl을 써도 마찬가지로 REST API 호출입니다.

Q. 전체적인 흐름(실무에서 어떻게 사용되는지, 왜 배워야 하는지)이 궁금합니다. MSA 이후 모듈러 모놀리식도 배우나요?

A. 실무에서 MSA가 어떻게 활용되고 왜 필요한지는 2번 섹션 "실무에서는 이렇게 활용된다"에서 다루고 있으니 참고해 주세요. MSA가 유일한 정답은 아니며, 조직·서비스 규모에 따라 모놀리식·모듈러 모놀리식·MSA 중에서 선택하는 것이 실무의 일반적인 접근입니다. SKALA 과정은 풀스택 개발자 양성 과정이라기 보다는, AI 서비스 엔지니어 양성과정이어서 AI 특성에 잘 맞는 MSA에 초점을 두고 있습니다.

Q. 스크럼(스프린트)을 나누는 기준이 무엇인가요? 스크럼을 통해 얻는 것은 무엇인가요? 팀원 역할에 따라 개인 과제 내용이 달라져도 되나요?

A. 6번 섹션에 자세히 설명해 두었습니다. 간단히 정리하면, 스프린트를 나누는 기준은 "이 기능이 없으면 서비스로서 가치를 줄 수 없는가?"이고, 스크럼을 통해 얻는 것은 빠른 피드백·리스크 조기 발견·유연한 우선순위 조정입니다. 팀원 역할에 따라 개인 과제 내용이 달라지는 것은 자연스러운 일이며, 각자의 결과물이 API 명세를 통해 하나의 스프린트 목표로 다시 연결되기만 하면 됩니다.

Docker 컨테이너 기초 가이드

질의 응답으로 배우는 핵심 개념 : 입문자용 개념 지도와 카테고리별 정리

스프링부트 및 MSA 개발 백엔드를 빌드·실행·배포하는 과정에서, 예상 질문 기반 재구성 학습 자료

전체 개념 지도 : 이 모든 질문은 어디서 출발하는가?

도커를 처음 접할 때 겪는 대부분의 혼란은, 사실 서로 다른 여러 개념이 아니라 단 하나의 뿌리에서 뻗어 나온 가지들입니다. 그 뿌리는 바로 "격리(Isolation)"입니다. 컨테이너는 기본적으로 호스트 컴퓨터와도, 다른 컨테이너와도 완전히 분리된 자기만의 세계에 있습니다. 지금부터 다루는 문법과 옵션들은 결국 "이 격리된 벽의 어디에, 어떻게 구멍을 뚫어 필요한 것만 연결할 것인가"를 다루는 도구들입니다.

이 원리를 먼저 이해하고 나면, 왜 WORKDIR가 호스트 폴더와 무관한지, 왜 포트를 두 번(호스트·컨테이너) 지정해야 하는지, 왜 컨테이너 여러 개를 묶으려면 별도의 네트워크 설정이 필요한지가 자연스럽게 설명됩니다. 이 문서는 그 뿌리에서 시작해서, 실제로 이미지를 만들고, 실행하고, 여러 개를 묶어 운영하는 순서로 구성되어 있습니다.

학습 흐름 5 단계

단계	이 단계에서 답하는 핵심 질문	관련 카테고리
1단계 · 개념의 뿌리	이미지와 컨테이너는 무엇이고 어떻게 다른가? 왜 "격리"되어 있다고 하는가?	I. 이미지의 기본 개념
2단계 · 이미지 만들기	Dockerfile의 각 명령어는 무엇을 하며, 왜 그 순서로 쓰는가?	II. Dockerfile 문법 III. 멀티 스테이지 빌드
3단계 · 이미지 실행하기	만든 이미지를 어떻게 실행하고, 외부와는 어떻게 연결하는가?	IV. docker run 옵션 V. 포트와 네트워크
4단계 · 여러 개 함께 운영	여러 컨테이너를 어떻게 하나의 서비스처럼 묶어 관리하는가?	VIII. Docker Compose
5단계 · 실전 관리	빌드 옵션, 민감 정보 관리, 문제 발생 시 내부를 어떻게 들여다보는가?	VI. build 옵션 VII. 환경변수 관리 IX. 컨테이너 내부 진입

참고: 이 문서의 각 장은 위 5단계 순서를 그대로 따릅니다. 처음 학습하신다면 반드시 I 장부터 순서대로 읽으시길 권장합니다. 이미 개념을 아신다면 궁금한 장으로 바로 이동하셔도 무방합니다.

카테고리별 요약표

본격적인 질의응답에 들어가기 전, 아홉 개 카테고리가 각각 무엇을 다루는지 한눈에 볼 수 있도록 정리합니다.

구분	카테고리	핵심 개념 요약	문항 수
I	이미지의 기본 개념	이미지란 무엇이며 컨테이너와 어떻게 다른가. 베이스 이미지와 태그의 신뢰 구조	9개
II	Dockerfile 문법	WORKDIR·COPY·RUN·USER 등 각 명령어가 실제로 하는 일	10개
III	멀티스테이지 빌드	빌드 전용 단계와 실행 전용 단계를 어떻게 분리하고 연결하는가?	6개
IV	docker run 명령어	컨테이너 이름·이미지 이름 구분, 옵션의 위치와 의미	8개
V	포트와 네트워크	호스트 포트와 컨테이너 포트를 왜 매핑해야 하는가?	3개
VI	docker build 옵션	이미지를 만들 때 쓰는 다양한 빌드 옵션 정리	3개
VII	환경변수와 보안	민감한 값을 명령어에서 분리해 안전하게 관리하는 법	2개
VIII	Docker Compose	여러 서비스를 하나의 파일로 정의하고 서로 연결하는 법	5개
IX	컨테이너 내부 진입	실행 중인 컨테이너에 들어가 문제를 진단하는 법	3개

I. 이미지(Image)의 기본 개념

도커를 배우는 첫걸음은 "이미지"와 "컨테이너"라는 두 단어를 정확히 구분하는 것입니다. 이 장에서는 이미지가 실제로 어디에 저장되는지, 왜 굳이 "이미지"라는 이름이 붙었는지, 그리고 베이스 이미지를 얼마나 신뢰할 수 있는지를 다룹니다.

질문 1 docker build로 이미지를 생성하면, 그 결과물은 실제 파일 시스템의 어느 위치에 저장되는가?

답변 이미지는 호스트의 눈에 보이는 폴더에 파일 하나로 생기지 않습니다. 도커 데몬이 관리하는 내부 저장소(리눅스 기준 /var/lib/docker, Mac/Windows의 Docker Desktop은 내부 리눅스 가상머신 안)에 여러 개의 레이어로 나뉘어 압축 저장됩니다.

평소에는 이 경로를 직접 다룰 필요가 없고, docker images 명령으로 목록을 확인하거나, 파일로 꺼내야 할 때만 docker save로 tar 파일을 별도로 만들면 됩니다.

```
docker images  
  
docker save -o my-app.tar my-app # 실제 파일(tar)로 내보내기
```

질문 2 개발을 처음 시작하는 사람에게 "이미지"라는 개념을 쉬운 비유로 설명한다면 어떻게 표현할 수 있는가?

답변 이미지는 "프로그램 실행에 필요한 모든 것을 통째로 담아 진공포장해 둔 밀키트"에 비유할 수 있습니다. 운영체제 기본 파일, 실행 환경(JRE), 우리가 만든 애플리케이션 코드, 실행 방법까지 전부 포함되어 있어서, 이 포장을 어디서 열어도(어떤 서버에서 실행해도) 항상 똑같이 동작합니다.

이미지는 읽기 전용이며 만들어진 뒤 내용이 바뀌지 않습니다. 반면 컨테이너는 그 밀키트를 실제로 꺼내 조리하고 있는, 살아 움직이는 상태를 가리킵니다. 같은 이미지 하나로 여러 개의 컨테이너를 동시에 만들어낼 수 있습니다.

질문 3 도커에서 "이미지(image)"라는 용어를 사용하는 이유는 무엇이며, 이 명칭은 어디서 유래했는가?

답변 이 용어는 도커가 처음 만든 말이 아니라, 컴퓨터 과학에서 오래전부터 쓰여온 "디스크 이미지", "VM 이미지"라는 개념을 그대로 계승한 것입니다. 사진을 찍으면 그 순간의 모습이 그대로 고정되어 나중에 봐도 똑같듯, 도커 이미지도 "이 프로그램을 실행하는 데 필요한 모든 파일과 설정을 특정 시점의 상태 그대로 고정해 저장해 둔 것"이라는 의미로 이 이름이 붙었습니다.

질문 4 이미지 대신 "설계도 파일"이라는 표현을 사용해도 의미상 문제가 없는가?

답변 엄밀히는 "설계도"라는 표현이 더 어울리는 대상은 이미지가 아니라 Dockerfile입니다.

Dockerfile은 "이렇게 만들어라"는 지시서일 뿐 아직 실체가 없는 반면, 이미지는 그 지시서대로 이미 조립과 포장까지 완전히 끝낸 완성품입니다.

정리하면 Dockerfile은 설계도, 이미지는 그 설계도로 이미 완성해 둔 결과물(밀키트), 컨테이너는 그 결과물을 실제로 실행 중인 상태로 3단계로 구분하는 것이 가장 정확합니다.

질문 5 Dockerfile의 FROM eclipse-temurin:21-jdk AS builder 구문은, 이미 존재하는 이미지를 기반으로 새 이미지를 만든다는 의미로 이해해도 되는가?

답변 그렇습니다. FROM은 이미 만들어져 있는 기존 이미지(여기서는 Eclipse Adoptium이 배포하는 공식 자바 이미지)를 가져와, 그 위에 새로운 내용을 층층이 쌓아 올리겠다는 선언입니다.

해당 이미지가 로컬에 없으면 도커가 자동으로 원격 저장소(도커허브 등)에서 내려받으며, 리눅스 설치나 자바 설치처럼 이미 검증된 작업을 처음부터 반복하지 않고 재사용할 수 있게 해 줍니다.

```
# 다음은 SpringBoot로 개발된 서비스를 상용 컨테이너로 빌드 배포하는 Dockerfile 예제
# 1단계 : Gradle로 Spring Boot 실행 JAR 빌드
# JDK = JRE + 개발 도구(javac, jar, javadoc 등) # 베이스 이미지가 eclipse-temurin:21-jdk가 설치된 스냅 샷이
# 며, 통상 실행 가능한 스냅 샷상태로 배포하므로 리눅스 OS를 포함

FROM eclipse-temurin:21-jdk AS builder

WORKDIR /workspace

COPY gradlew ./
COPY gradle ./gradle
COPY settings.gradle ./
COPY build.gradle ./

RUN chmod +x ./gradlew

COPY src ./src

RUN ./gradlew clean bootJar \
    --no-daemon \
    --no-configuration-cache
```

```

# 2단계 : 실행 이미지
# 베이스 이미지가 eclipse-temurin:21-jre가 설치된 스냅 샷이며, 실행한 필요한 jre로만 구성됨
FROM eclipse-temurin:21-jre

WORKDIR /app

RUN groupadd --system spring ₩
    && useradd --system ₩
        --gid spring ₩
        --create-home ₩
        spring

COPY --from=builder ₩
    /workspace/build/libs/*.jar ₩
    /app/application.jar

RUN chown spring:spring /app/application.jar

USER spring:spring

EXPOSE 9999

ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-Djava.security.egd=file:/dev/./urandom", "-jar",
"/app/application.jar"]

```

질문 6 베이스 이미지 안에 리눅스 관련 파일을 별도로 추가하지 않아도, 컨테이너는 리눅스 기반 이므로 당연히 포함되어 있다고 보아도 되는가?

답변 정확한 이해가 필요합니다. 컨테이너는 가상머신과 달리 리눅스 커널을 이미지 안에 포함하지 않고, 컨테이너를 실행하는 호스트(또는 Docker Desktop이 띄우는 내부 리눅스 가상머신)의 커널을 그대로 빌려 씁니다.

반면 셸이나 기본 유틸리티 같은 최소한의 리눅스 파일들은 베이스 이미지 제작자가 명시적으로 포함시켜 둔 것입니다. FROM scratch처럼 정말 아무것도 없는 이미지도 존재하므로, "당연히 자동으로 포함된다"기보다는 "베이스 이미지 제작자가 의도적으로 넣어 두었다"는 표현이 더 정확합니다.

질문 7 이미지 태그(예: -jammy, -alpine)는 실제 내용물과 항상 일치한다고 신뢰할 수 있는가?

답변 아닙니다. 태그는 이미지 제작자가 자율적으로 붙이는 문자열일 뿐, 도커가 내용물과 태그의 일치 여부를 검증해 주지 않습니다. 만드는 사람이 태그를 잘못 붙이거나 아예 생략해도 빌드 자체는 문제없이 진행됩니다.

그래서 실무에서는 태그 이름보다 "누가 만들었는가"(Docker Hub의 Official Image-Verified Publisher 표시)를 신뢰의 기준으로 삼고, 정말 중요한 경우에는 태그 대신 절대 변하지 않는 digest(해시값)로 이미지를 고정해서 사용합니다.

```
eclipse-temurin@sha256:abcd1234... # 태그 대신 digest로 고정하는 예
```

질문 8 특별한 사유가 없다면, Dockerfile은 리눅스가 이미 포함된 베이스 이미지를 기준으로 작성하는 것이 일반적인 관례라고 보아도 되는가?

답변 그렇습니다. 리눅스는 가볍고 무료이며, 도커 자체가 리눅스 컨테이너 기술에서 출발한 도구라 생태계 전체가 리눅스 중심으로 발전해 왔습니다.

다만 예외도 있습니다. 윈도우 전용 애플리케이션에는 별도의 윈도우 기반 베이스 이미지가 쓰이며, Go 언어처럼 외부 의존성이 거의 없는 정적 바이너리는 완전히 빈 이미지인 FROM scratch에서 시작하기도 합니다.

질문 9 JDK 이미지와 JRE 이미지는 어떤 기준으로 구분해서 사용해야 하는가?

답변 JRE(Java Runtime Environment)는 자바 프로그램을 실행만 하는 데 필요한 최소 환경이고, JDK(Java Development Kit)는 그 JRE 전체에 컴파일러(javac)-jar 패키징 도구 같은 개발 도구가 추가된 것입니다. 즉 $JDK = JRE + \text{개발 도구}$ 입니다.

이 때문에 소스코드를 컴파일하는 빌드 단계는 JDK가 필요하지만, 이미 컴파일이 끝난 jar 파일을 실행만 하는 단계는 JRE만으로 충분합니다. JRE는 개발 도구가 빠진 만큼 이미지 용량이 작고 불필요한 도구로 인한 보안 위험도 줄어들기 때문에, 실행 전용 이미지는 굳이 JDK를 쓰지 않고 JRE로 구성하는 것이 표준적인 최적화 방법입니다.

II. Dockerfile 문법 파헤치기

이 장에서는 실제 Spring Boot 프로젝트의 Dockerfile을 한 줄씩 뜯어보며, WORKDIR·COPY·RUN·USER 같은 각 명령어가 정확히 무엇을 하는지 확인합니다. 특히 "이 경로가 내 컴퓨터의 경로인가, 컨테이너 안의 경로인가"를 구분하는 것이 이 장의 핵심입니다.

질문 1 Dockerfile의 WORKDIR /workspace는 호스트 컴퓨터의 실제 폴더 위치를 가리키는 것인가?

답변 아닙니다. WORKDIR는 이미지(그리고 이후 컨테이너) 내부에만 존재하는 가상의 작업 디렉터리를 지정합니다. 해당 경로가 이미지 안에 아직 없으면 도커가 자동으로 만들어 주며, 이후 나오는 상대 경로 명령어들은 모두 이 디렉터리를 기준으로 동작합니다.

호스트 컴퓨터 어디를 찾아봐도 이 폴더는 존재하지 않으며, 오직 이미지·컨테이너의 격리된 파일 시스템 안에서만 의미를 가집니다.

질문 2 실행 중인 컨테이너 내부에 접속했을 때 해당 작업 폴더가 보이지 않는 경우, 어떤 이유를 의심해 볼 수 있는가?

답변 이 프로젝트처럼 멀티스테이지 빌드 구조라면 지극히 정상입니다. /workspace는 1단계(builder)에서만 사용되는 임시 작업 공간이며, 최종적으로 실행되는 2단계 이미지는 완전히 다른 베이스에서 새로 시작되기 때문에 1단계의 폴더를 물려받지 않습니다.

실제로 실행 중인 컨테이너 내부를 확인해 보면 /workspace가 아니라 2단계에서 지정한 /app 폴더 안에 애플리케이션 jar 파일만 존재하는 것을 볼 수 있습니다.

```
docker exec -it 컨테이너이름 /bin/sh
ls /workspace # No such file or directory
ls /app      # application.jar 만 보임
```

질문 3 COPY gradlew ./ 명령어에서 첫 번째 인자와 두 번째 인자는 각각 무엇을 의미하는가?

답변 COPY는 항상 "출발지 목적지" 순서를 가집니다. 첫 번째 값(gradlew)은 빌드 컨텍스트, 즉 호스트의 현재 프로젝트 폴더 안에 있는 파일을 가리키고, 두 번째 값(.)은 이미지 내부의 목적지 경로를 가리킵니다. 여기서 마침표(.)는 앞서 WORKDIR로 지정해 둔 현재 작업 디렉터리를 뜻합니다. 결과적으로 호스트의 ./gradlew 파일이 이미지 내부의 /workspace/gradlew 위치로 복사되는 것입니다.

질문 4 RUN `chmod +x ./gradlew` 명령어는 파일에 실행 권한을 부여하는 리눅스 명령으로 이해하면 되는가?

답변 맞습니다. `chmod`는 `change mode`의 약자로 파일·디렉터리의 권한을 바꾸는 명령이며, `+x`는 실행(`execute`) 권한을 추가한다는 뜻입니다. 리눅스는 파일을 복사할 때 실행 권한이 유지되지 않는 경우가 있어, `gradlew`처럼 직접 실행해야 하는 스크립트에는 이 권한을 명시적으로 부여해 주어야 합니다.

질문 5 RUN `./gradlew clean bootJar` 명령은 호스트가 아니라 컨테이너(빌드용 임시 컨테이너) 내부에서 실행되는 것이 맞는가?

답변 맞습니다. RUN은 이미지를 빌드하는 과정에서 도커가 잠깐 띄우는 빌드 전용 컨테이너 내부에서 실행됩니다. 앞서 COPY로 옮겨 둔 `/workspace` 안의 파일들을 이용해 그 컨테이너 안에서 실제로 빌드가 이루어지며, 빌드가 끝나면 결과물(`jar` 파일)만 다음 레이어에 남고 그 임시 컨테이너 자체는 최종 이미지에 포함되지 않습니다.

질문 6 2단계에서 다시 등장하는 `WORKDIR /app` 역시 호스트 경로를 가리키는가?

답변 아닙니다. 이는 1단계의 `WORKDIR /workspace`와 완전히 같은 개념이지만, 서로 다른 이미지(스테이지) 안에 각각 독립적으로 존재하는 경로입니다. 두 스테이지는 FROM으로 완전히 새로 시작되는 별개의 이미지이므로, 1단계의 `/workspace`와 2단계의 `/app`은 이름이 우연히 다를 뿐 서로 아무런 관련이 없습니다.

질문 7 `groupadd --system spring` 및 `useradd` 명령으로 구성된 블록은 정확히 어떤 작업을 수행하는가?

답변 이 두 명령은 `root`가 아닌 별도의 실행 전용 계정(`spring`)을 새로 만드는 작업입니다. `groupadd --system spring`은 서비스 실행 전용 시스템 그룹을 생성하고, `useradd --system --gid spring --create-home spring`은 그 그룹에 속한 시스템 사용자 계정을 만들며 홈 디렉터리도 함께 생성합니다. 도커 컨테이너는 기본적으로 `root` 권한으로 실행되는데, 만약 애플리케이션이 `root` 권한으로 동작하다가 취약점을 통해 뚫리면 공격자가 컨테이너 안에서 `root` 권한을 그대로 얻게 됩니다. 이를 방지하기 위해 권한이 제한된 전용 계정을 미리 만들어 두는 최소 권한 원칙에 따른 보안 조치입니다.

질문 8 RUN `chown spring:spring /app/application.jar` 와 USER `spring:spring` 두 줄은 각각 무

엇을 하며, 왜 이 순서로 배치되어 있는가?

답변 chown(change owner)은 파일의 소유자를 바꾸는 명령으로, COPY 직후 root 소유로 복사된 jar 파일의 소유권을 앞서 만든 spring 계정으로 넘겨줍니다. 이어지는 USER spring:spring은 이 시점부터 이후의 모든 명령어(그리고 최종적으로 컨테이너 실행 자체)가 spring 계정 권한으로 수행되도록 전환하는 지시어입니다.

순서가 중요한 이유는, chown처럼 다른 사용자에게 파일 소유권을 넘기는 작업은 root 권한이 있어야만 가능하기 때문입니다. 만약 USER spring:spring을 먼저 선언한 뒤 chown을 실행하면, 권한이 없는 spring 계정이 그 작업을 수행할 수 없어 오류가 발생합니다.

질문 9 ENTRYPOINT에 지정된 java 실행 옵션들, 특히 -XX:MaxRAMPercentage와 -Djava.security.egd는 각각 어떤 역할을 하는가?

답변 -XX:MaxRAMPercentage=75.0은 컨테이너에 할당된 전체 메모리 중 최대 75퍼센트까지만 JVM 힙 메모리로 사용하도록 제한하는 옵션입니다. 컨테이너의 메모리 한도를 JVM이 인식하지 못하면 과도하게 힙을 잡아 컨테이너가 강제 종료(OOM)될 수 있어, 컨테이너 환경에서 특히 중요한 설정입니다.

-Djava.security.egd=file:/dev/./urandom은 SecureRandom이 난수를 생성할 때 참조하는 엔트로피 소스를 지정하는 옵션입니다. 기본값인 /dev/random은 엔트로피가 부족한 컨테이너 환경에서 응답을 기다리며 앱 시작이 지연될 수 있는데, 이를 방지하기 위해 블로킹 없이 빠르게 응답하는 /dev/urandom을 사용하도록 강제합니다.

EGD는 Entropy Gathering Daemon(엔트로피 수집 데몬)의 약자이며, 경로 중간에 붙은 마침표(/dev/./urandom)는 일부 JVM 버전이 "/dev/random"이라는 문자열을 하드코딩으로 특별 취급하는 알려진 버그를 우회하기 위한 관례적인 트릭입니다.

질문 10 이미지의 준비 단계(계정 생성 등)는 root로 진행하면서, 정작 애플리케이션 실행은 spring 계정으로 전환하는 이유는 무엇인가? 처음부터 spring 계정으로 진행하면 안 되는 이유가 있는가?

답변 groupadd, useradd, chown 같은 명령어는 리눅스 권한 체계상 반드시 root(관리자) 권한이 있어야만 실행할 수 있습니다. spring 계정 자체를 아직 만들지 않은 시점에는, spring으로 로그인하려 해도 계정이 존재하지 않으므로 애초에 그 계정으로 새 계정을 만들거나 파일 소유권을 넘기는 관리 작업을 수행할 권한이 없습니다.

그래서 root로 시작해 계정 생성·소유권 이전 같은 준비 작업을 마친 뒤, 실제로 외부 요청을 처리하며 공격에 노출될 수 있는 애플리케이션 실행 구간만 USER 지시어로 spring 계정으로 전환하는 것이 유일하게 가능한 순서입니다. 보안에서 중요한 것은 "누가 이미지를 만들었는가"가 아니라 "실제 서비스가 돌아가는 동안 어떤 권한으로 실행되는가"이므로, 이 설계는 그 원칙에 정확히 부합합니다.

III. 멀티스테이지 빌드의 원리

하나의 Dockerfile 안에 FROM이 두 번 등장하는 멀티스테이지 빌드는, 무거운 빌드 도구와 소스코드를 최종 이미지에서 제외하기 위한 핵심 기법입니다. 이 장에서는 두 단계가 실제로 어떻게 연결되고, 왜 이 방식이 안전하게 작동하는지를 다룹니다.

질문 1 COPY --from=builder 구문은 정확히 어떤 동작을 수행하는가?

답변 일반적인 COPY는 호스트에서 파일을 가져오지만, --from=builder가 붙으면 호스트가 아니라 앞서 AS builder로 이름 붙여 둔 1단계 이미지에서 파일을 가져오라는 지시가 됩니다.

이는 "이미 완성된 다른 이미지에서 필요한 재료 하나만 꺼내 와, 지금 만들고 있는 새 이미지에 얹는 것"에 비유할 수 있습니다. 나머지 재료(소스코드, JDK, gradle 캐시)는 그대로 1단계 이미지에 남겨 둔 채 손대지 않습니다.

질문 2 빌드 결과물인 jar 파일이 build/libs 폴더 안에 여러 개 존재하는 경우, COPY --from=builder ... *.jar ... application.jar 구문은 어떻게 동작하는가?

답변 도커의 COPY 규칙상, 와일드카드로 여러 파일이 매칭되는 경우 목적지는 반드시 디렉터리여야 하며 끝에 슬래시(/)가 있어야 합니다. 지금처럼 목적지가 application.jar라는 특정 파일명으로 고정되어 있으면, 두 개 이상의 파일이 매칭되는 순간 도커가 어느 파일을 그 이름으로 써야 할지 판단하지 못해 빌드가 실패합니다.

실제로 Spring Boot Gradle 플러그인은 기본 설정에서 실행 가능한 bootJar와 의존성이 빠진 plain jar 두 가지를 동시에 만들 수 있어, build 태스크 전체를 실행하면 이 문제가 재현될 수 있습니다.

질문 3 그렇다면 정확한 파일명 대신 와일드카드(*.jar)를 사용한 것은 실수이거나, 파일이 하나만 있을 것이라고 막연히 추정한 결과인가?

답변 실수가 아니라 의도적인 설계에 가깝습니다. 실제 jar 파일명은 build.gradle에 적힌 프로젝트 이름과 버전에 따라 계속 달라지므로, 이를 Dockerfile에 하드코딩해 두면 버전이 바뀔 때마다 Dockerfile도 함께 수정해야 하는 번거로움이 생깁니다.

이 프로젝트는 RUN ./gradlew clean bootJar처럼 bootJar 태스크만 명시적으로 실행하도록 지정해, build/libs 폴더 안에 항상 정확히 하나의 jar만 생성되도록 전제 조건을 먼저 맞추어 두었습니다. 그 전제 위에서만 와일드카드가 안전하게 동작하는 구조입니다.

질문 4 이 명령어에서 `clean` 태스크는 이름이 같은 이전 파일만 선택적으로 삭제하는 것인가, 아니면 해당 폴더의 내용을 통째로 삭제하는 것인가?

답변 `clean`은 이름이 같은 파일만 골라 지우는 것이 아니라, 프로젝트의 `build` 폴더 자체를 통째로 삭제합니다. 그 안에 있던 `jar` 파일, 컴파일된 클래스 파일, 임시 파일 등이 무엇이든 상관없이 폴더째로 제거되며, 이후 `bootJar`가 처음부터 다시 필요한 파일을 새로 만들어냅니다.

특정 파일만 선별해서 지우려면 최신·과거 버전을 판별하는 복잡한 로직이 필요하지만, 폴더 전체를 밀어버리고 새로 시작하는 방식은 훨씬 단순하고 예외 상황을 원천 차단할 수 있다는 장점이 있습니다.

질문 5 `build.gradle`의 버전 값이 계속 바뀐다면, 이전 버전의 `jar` 파일과 새 버전의 `jar` 파일이 `build/libs` 폴더 안에 함께 쌓이게 되는 것은 아닌가?

답변 쌓이지 않습니다. 두 가지 안전장치가 겹쳐 있기 때문입니다. 첫째, `gradlew clean`이 매 빌드 시작 전 `build/libs`를 포함한 `build` 폴더 전체를 삭제합니다. 둘째, 도커의 멀티스테이지 빌드는 매번 완전히 새로운 컨테이너(빈 파일시스템)에서 시작하므로, 애초에 이전 빌드의 흔적이 그 컨테이너 안에 물리적으로 존재하지 않습니다.

결과적으로 버전이 계속 바뀌더라도, 빌드가 끝난 시점의 `build/libs` 폴더 안에는 그 순간의 버전으로 만들어진 `jar` 파일 하나만 항상 존재하게 됩니다.

질문 6 1단계에서 `COPY`와 빌드까지 마친 소스코드와 `JDK`는, 정확히 어느 시점에 어떤 방식으로 최종 이미지에서 제외되는가?

답변 "제외된다"보다 정확한 표현은 "애초에 포함된 적이 없다"입니다. 두 번째 `FROM`이 등장하는 순간, 도커는 1단계와 완전히 무관한 새로운 베이스(`eclipse-temurin:21-jre`)에서 이미지를 처음부터 새로 쌓기 시작합니다. 1단계에서 쌓아 온 레이어(`JDK`, 소스코드, `gradle` 캐시)는 이 새로운 이미지의 계보에 한 번도 속한 적이 없습니다.

1단계 결과물은 도커의 로컬 빌드 캐시 안에 이름 없는 임시 이미지로만 남고, 유일하게 최종 이미지로 넘어오는 것은 `COPY --from=builder`로 명시한 `jar` 파일 하나뿐입니다. 즉 어느 시점에 지워지는 것이 아니라, 처음부터 서로 다른 두 개의 독립된 이미지이며 그중 하나에서 파일 하나만 명시적으로 옮겨 오는 구조라는 점이 핵심입니다.

참고: 최종 이미지의 실제 레이어 구성은 (1) `JRE` 베이스 (2) `WORKDIR /app` (3) `spring` 계정 생성 (4) `COPY`로 가져온 `jar` 파일 (5) `chown USER:ENTRYPOINT` 메타데이터, 이렇게 다섯 가지뿐이며 소스코드

드나 JDK는 이 목록 어디에도 없습니다.

IV. docker run 명령어와 실행 옵션

이미지를 실제로 실행할 때 사용하는 docker run 명령어는 옵션이 많아 처음에는 복잡해 보입니다. 이 장에서는 각 옵션의 의미와, 특히 자주 헷갈리는 "컨테이너 이름"과 "이미지 이름"의 구분을 정리합니다.

질문 1 `docker save -o 파일명 이미지명` 명령에서, 출력 파일을 뜻하는 `-o` 옵션이 이미지 이름보다 앞에 오는 이유는 무엇인가? 통상적인 좌에서 우로 읽는 흐름과 다르게 느껴진다.

답변 이는 옵션의 위치가 고정된 규칙이 아니기 때문입니다. `-o`는 옵션(플래그)이고 이미지 이름은 위치 인자(positional argument)인데, 대부분의 명령줄 도구는 옵션을 어느 위치에 두어도 동일하게 동작하도록 설계되어 있습니다.

`cp source dest`처럼 두 값의 순서 자체가 의미(원본에서 대상으로)를 가지는 명령과 달리, `docker save`는 `-o`라는 옵션이 목적지를 명시적으로 지정하므로 순서가 아니라 `-o` 뒤에 오는 값이 목적지임을 나타냅니다. 실제로 `docker save 이미지명 -o 파일명` 순서로 써도 동일하게 동작합니다.

```
docker save -o app.tar my-app
docker save my-app -o app.tar # 순서를 바꿔도 동일하게 동작
```

질문 2 `-o` 옵션의 정확한 명칭과 그 의미는 무엇인가?

답변 `-o`는 output의 약자로, 뒤에 오는 값이 결과물을 저장할 출력 파일명임을 지정합니다. 대부분의 도커 명령어에서 `-o`와 `--output`은 동일한 의미로 함께 제공됩니다.

질문 3 `docker run` 명령어에서 `--name` 뒤에 오는 값은 컨테이너의 이름을 지정하는 것인가, 이미지의 이름을 지정하는 것인가?

답변 `--name`은 컨테이너의 이름을 지정하는 옵션입니다. 이미지 이름은 별도의 플래그 없이, 모든 옵션이 끝난 뒤 남는 위치 인자(대개 명령어의 맨 끝)로 지정됩니다.

두 값을 동일한 문자열로 지어 두면 구분이 어려워 보일 수 있지만, `docker images`로는 이미지 목록을, `docker ps`로는 컨테이너 목록을 확인해 보면 이 둘이 서로 별개의 개념임을 명확히 확인할 수 있습니다.

```
docker run -d --name my-container my-image
```

명령어 구성 요소	의미
--name my-container	컨테이너 이름 지정 (docker ps 로 확인)
my-image (맨 끝, 옵션 없는 위치 인자)	실행할 이미지 이름 지정 (docker images 로 확인)

질문 4 이미지 이름을 지정하는 전용 옵션(플래그)이 별도로 존재하는가?

답변 존재하지 않습니다. docker run [옵션들] 이미지 [실행할 명령] 이라는 명령어 구조 자체가 고정되어 있어서, 이미지는 옵션이 아니라 반드시 있어야 하는 필수 위치 인자로 지정하도록 설계되어 있습니다.

docker build의 -t(태그) 옵션과 혼동하기 쉬운데, build는 "새 이미지를 만들며 이름을 짓는" 작업이라 옵션이 필요하지만, run은 "이미 존재하는 이미지 중 무엇을 실행할지" 위치로만 알려주면 충분하기 때문에 이런 차이가 생깁니다.

질문 5 --add-host host.docker.internal:host-gateway 옵션은 어떤 문제를 해결하기 위한 설정인가?

답변 컨테이너 안에서 자신을 실행하고 있는 호스트 머신 자체를 가리킬 수 있도록, host.docker.internal이라는 이름을 호스트의 게이트웨이 IP로 매핑해 주는 설정입니다.

이 프로젝트는 컨테이너 안의 스프링 앱이 호스트 Mac에서 실행 중인 MariaDB(host.docker.internal:53301)에 접속해야 하는데, 컨테이너는 원래 격리되어 있어 호스트를 직접 알 수 없기 때문에 이 설정이 필요합니다.

질문 6 이 host-gateway 설정은 Mac이나 Windows 환경에 국한된 것이며, 리눅스 환경에서는 다른 방식이 필요한가?

답변 실제로는 정반대에 가깝습니다. Mac·Windows의 Docker Desktop에서는 host.docker.internal 이 원래 자동으로 지원되지만, 리눅스에는 이 이름 자체가 기본적으로 존재하지 않습니다.

Docker 20.10부터 host-gateway라는 예약어가 도입되어, --add-host로 직접 등록하면 리눅스에서도 동일한 이름을 쓸 수 있게 되었습니다. 즉 이 한 줄은 Mac에서 개발한 설정을 리눅스 서버에 그대로 배포해도 동작하게 만들기 위한 이식성 확보용 코드입니다.

질문 7 이 한 줄이 없어도 Mac이나 Windows에서는 정상적으로 동작하는 것으로 이해해도 되는

가?

답변 그렇습니다. Docker Desktop은 host.docker.internal을 자동으로 지원하므로, 이 설정이 없어도 Mac·Windows에서는 문제없이 동작합니다. 이 설정은 오직 향후 리눅스 서버로 배포할 가능성까지 대비한 안전장치입니다.

질문 8 긴 docker run 명령어를 여러 줄로 나누어 작성할 때, 옵션 사이에 주석(#)이나 빈 줄을 넣어도 문제없이 실행되는가?

답변 실행되지 않습니다. 줄 끝의 역슬래시(라인 연속 문자)는 다음 줄과 하나로 이어 붙이는 역할을 하는데, 이는 바로 다음 문자가 실제 줄바꿈일 때만 성립합니다. 그런데 주석(#)을 만나면 셸은 그 줄의 # 이후 내용을 전부 무시해 버리므로, 그 줄이 역슬래시로 끝나 있었다 해도 더 이상 줄 잇기 기능을 하지 못하고 명령어가 그 지점에서 끊깁니다. 빈 줄도 마찬가지로 역슬래시가 없으므로 이어붙이기가 끊깁니다.

실제로 아래와 같이 테스트해 보면, 첫 줄 명령까지만 실행되고 그 다음 줄부터는 command not found 오류를 내는 별개의 잘못된 명령으로 처리되는 것을 확인할 수 있습니다.

```
echo start ₩
# 이 주석이 라인 연속을 끊어버림
--name foo
```

결과: "start"만 출력되고, --name foo: command not found 오류 발생

참고: 여러 줄 명령어에 설명을 남기고 싶다면, 명령어 앞뒤에 별도의 주석 줄을 두거나 값들을 변수로 뽑아 별도의 셸 스크립트로 작성하는 방식을 사용해야 합니다.

V. 포트와 네트워크 연결

컨테이너와 외부를 연결하는 -p 옵션은 격리 개념이 가장 직접적으로 드러나는 지점입니다. 왜 포트를 두 번이나 지정해야 하는지, 그 이유를 이 장에서 확인합니다.

질문 1 docker run -p 9999:9999처럼 호스트 포트를 지정하는 것이 실제로 의미가 있는가? 컨테이너가 요청을 직접 받아야 하는 것은 아닌가?

답변 컨테이너는 기본적으로 호스트와 분리된 자기만의 가상 네트워크 공간에 있어서, 별도의 매핑 없이는 호스트나 외부에서 컨테이너 내부로 직접 접근할 방법이 없습니다. 그래서 호스트 포트가 반드시 필요합니다.

-p 9999:9999는 "호스트의 9999번 포트에 들어오는 트래픽을 컨테이너 내부의 9999번 포트에 전달해 달라"는 명시적인 다리 놓기 작업입니다. 이 매핑이 없으면 컨테이너 안에서 앱이 아무리 잘 동작해도 호스트에서는 그 존재조차 알 수 없습니다. 이렇게 격리해 두는 이유는, 여러 컨테이너가 내부적으로 같은 포트(예: 8080)를 동시에 사용하더라도 호스트 쪽 매핑 번호만 다르게 지정하면 포트 충돌 없이 동시에 운영할 수 있기 때문입니다.

질문 2 결국 외부에서 접속하는 포트와 컨테이너 내부 포트를 서로 매핑해 주어야 한다는 뜻으로 이해하면 되는가?

답변 정확합니다. -p 옵션의 앞쪽 값(호스트 포트)이 곧 외부에서 접속할 때 사용하는 창구이며, 뒤쪽 값(컨테이너 포트)이 실제 애플리케이션이 리스닝하는 지점입니다. 이 둘을 명시적으로 연결해 주는 것이 -p 옵션의 역할입니다.

질문 3 호스트 포트라는 것은 결국 내 컴퓨터(호스트)에서 열어 둔 포트이므로, 외부에서 접속 가능한 포트라는 의미로 받아들여도 되는가?

답변 기본적으로는 맞지만, "외부"의 범위를 두 단계로 나누어 이해할 필요가 있습니다. 호스트 포트가 열려 있으면 컨테이너 바깥, 즉 호스트 자체나 같은 네트워크(같은 Wi-Fi)의 다른 기기에서는 확실히 접속할 수 있습니다. 그러나 전 세계 인터넷 어디서든 접속 가능한 상태가 되려면, 호스트 포트 개방에 더해 공유기의 포트포워딩이나 ngrok 같은 터널링 설정이 한 단계 더 필요합니다. "컨테이너 밖에서 접속 가능하게 하는 문"이라는 이해는 정확하지만, 그 문이 인터넷 전체에도 열려 있는지는 별개의 문제입니다.

VI. docker build 옵션 정리

이미지를 만드는 docker build 명령에도 다양한 옵션이 있습니다. 자주 쓰는 -t 옵션을 시작으로, 나머지 옵션들과 build·run 사이의 역할 구분을 정리합니다.

질문 1 docker build -t my-app . 명령에서 -t 옵션은 무엇을 지정하는가?

답변 -t는 tag의 약자로, 빌드한 이미지에 이름(과 선택적으로 버전 태그)을 붙이는 옵션입니다. 태그를 생략하면 자동으로 latest가 붙어 my-app:latest로 저장됩니다.

만약 -t를 전혀 쓰지 않으면 이미지는 이름 없이(<none>) 만들어져, 오직 이미지 ID(해시값)로만 구분해야 하므로 실무에서는 사실상 항상 지정하는 필수 옵션입니다.

```
docker build -t my-app .  
docker build -t my-app:1.0 . # 버전 태그까지 명시
```

질문 2 -t 이외에 docker build에서 자주 사용되는 다른 옵션들에는 어떤 것이 있는가?

답변 -f(--file)는 기본값이 아닌 다른 이름의 Dockerfile을 지정할 때, --no-cache는 이전 레이어 캐시를 무시하고 처음부터 다시 빌드할 때, --build-arg는 빌드 시점에만 쓰이는 변수를 Dockerfile의 ARG에 전달할 때 사용합니다.

이 밖에 --pull(베이스 이미지를 최신으로 갱신), --platform(대상 CPU 아키텍처 지정, 예: Mac에서 서버용 이미지 빌드 시), --target(멀티스테이지 중 특정 단계까지만 빌드), -q(조용히 실행 후 이미지 ID만 출력) 등이 자주 쓰입니다.

```
docker build --no-cache -t loginauth-extended-app .  
docker build --platform linux/amd64 -t my-app .
```

질문 3 docker build 단계에도 포트를 지정하는 옵션이 존재하는가?

답변 존재하지 않습니다. 빌드는 이미지를 조립하는 단계일 뿐 네트워크 서비스를 실제로 띄우는 단계가 아니므로, 포트 매핑은 오직 docker run에서만 의미를 가집니다.

참고로 docker build에는 --network라는 옵션이 있지만, 이는 포트 개방과 무관하게 빌드 중(RUN 명령 실행 시) 라이브러리 다운로드 등에 필요한 네트워크 접근 방식을 지정하는 별개의 기능입니다.

VII. 환경변수와 보안 관리

데이터베이스 비밀번호나 JWT 시크릿처럼 민감한 값을 docker run 명령어에 그대로 나열하면, 셸 히스토리나 화면 공유 등을 통해 노출될 위험이 있습니다. 이 장에서는 이를 안전하게 분리하는 방법을 다룹니다.

질문 1 docker run 명령어에 비밀번호나 시크릿 값을 -e 옵션으로 직접 나열하면 그대로 노출되는데, 이를 분리해서 실행하는 방법은 무엇인가?

답변 별도의 환경변수 파일을 만들고 --env-file 옵션으로 불러오는 방식을 사용합니다. 파일 안에 KEY=VALUE 형태로 값을 나열해 두면, 명령어 자체에는 민감한 값이 전혀 노출되지 않습니다.

이 파일은 반드시 .gitignore에 등록해 깃허브 등에 실수로 올라가지 않도록 해야 하며, 팀원과 형식을 공유할 때는 실제 값이 빠진 .env.example 같은 템플릿 파일을 별도로 관리하는 것이 일반적입니다. 다만 이 방법도 컨테이너 실행 후 docker inspect로는 여전히 값이 노출되므로, 정말 엄격한 보안이 필요하다면 Docker Secrets나 클라우드의 Secrets Manager 같은 별도 도구를 고려해야 합니다.

```
# .env
DB_URL=jdbc:mariadb://host.docker.internal:53301/sql_db
DB_PASSWORD=SqlDbA-1
JWT_SECRET=change-this-development-secret-key

docker run -d --env-file .env my-app
```

질문 2 macOS에서는 .env 파일이 이름 앞의 마침표 때문에 자동으로 숨김 파일 처리되어 불편한데, 마침표 없이 env라는 이름으로 사용해도 무방한가?

답변 무방합니다. .env라는 이름은 강제되는 규칙이 아니라 관례일 뿐이며, --env-file 옵션에 원하는 파일명을 그대로 지정하면 동일하게 동작합니다.

다만 docker compose 자체는 프로젝트 폴더에 정확히 .env라는 이름의 파일이 있을 경우, docker-compose.yml 파일 안에서 변수를 치환하는 별도의 자동 인식 기능을 제공합니다. 이 기능까지 활용할 계획이 없다면 env처럼 원하는 이름으로 자유롭게 바꾸어도 문제없습니다.

```
docker run -d --env-file env my-app
```

VIII. Docker Compose 와 다중 서비스 구성

여러 컨테이너를 하나의 파일로 정의하고 함께 운영하는 Docker Compose는, 특히 마이크로서비스 처럼 서비스가 여러 개로 늘어날 때 진가를 발휘합니다. 이 장에서는 compose 문법 기초부터, 서비스 폴더가 여러 개일 때 이미지가 어떻게 만들어지고 서로 연결되는지까지 다룹니다.

질문 1 `docker-compose.yml`의 `build` 항목에서 `context: .` 와 `dockerfile: Dockerfile` 두 줄은, 현재 위치에 있는 `Dockerfile`로 빌드하라는 의미로 이해하면 되는가?

답변 정확합니다. `context: .`는 빌드에 사용할 기준 디렉터리(빌드 컨텍스트)를 `docker-compose.yml`이 있는 현재 위치로 지정하고, `dockerfile: Dockerfile`은 그 컨텍스트 안에서 사용할 `Dockerfile`의 파일명을 지정합니다.

`Dockerfile` 안의 `COPY` 같은 명령어들은 모두 이 컨텍스트를 기준으로 파일을 찾으므로, 이 두 줄은 결국 `docker build -f Dockerfile .` 명령과 동일한 의미를 가집니다.

질문 2 `Dockerfile`이라는 이름이 기본값이라면, `dockerfile` 항목 자체를 생략해도 되는가?

답변 파일명이 정확히 `Dockerfile`이고 위치가 `context` 디렉터리의 최상위라면 생략할 수 있습니다. 나아가 `context`도 기본값인 현재 디렉터리(.)이므로, `build: .` 한 줄로 더 축약할 수도 있습니다. 반대로 파일명이 `Dockerfile.prod`처럼 다르거나 하위 폴더에 위치한다면, 반드시 `dockerfile` 항목으로 명시해 주어야 합니다.

```
# 원래 표기
build:
  context: .
  dockerfile: Dockerfile

# 축약 가능
build: .
```

질문 3 자바 코드가 담긴 여러 개의 마이크로서비스 폴더(`eureka-server`, `auth-server`, `api-gateway`, `user-service` 등)를 각각 `build: context`로 지정한 하나의 `docker-compose.yml`을 실행하면, 이미지는 폴더 개수만큼 각각 생성되는가, 아니면 최종적으로 하나의 이미지로 통합되는가?

답변 폴더(서비스) 개수만큼 완전히 독립된 이미지가 각각 생성됩니다. `build` 항목이 있는 서비스마다 `context`가 서로 다른 폴더를 가리키므로, 도커는 이를 각각 별개의 소스와 별개의 `Dockerfile`로 진행하는 완전히 다른 빌드로 인식합니다.

반면 mariadb, kafka처럼 build 없이 image 항목만 있는 서비스는 새로 빌드되지 않고 이미 만들어진 이미지를 원격 저장소에서 그대로 내려받기만 합니다. 예시로 든 compose 파일 기준으로는 build가 있는 8개 서비스가 각각의 이미지로 빌드되고, 여기에 미리 만들어진 이미지 2개(mariadb, kafka)가 더해져 총 10개의 이미지가 만들어집니다.

참고: 서비스별로 이미지가 독립적으로 생성되는 이 구조 덕분에, 예를 들어 user-service 코드만 수정해도 그 이미지만 다시 빌드하면 되고 다른 서비스의 이미지는 전혀 영향을 받지 않습니다. 이는 마이크로서비스가 지향하는 "서비스별 독립 배포" 원칙이 그대로 구현된 형태입니다.

질문 4 이렇게 독립적으로 생성된 이미지들이라도, 실행되었을 때는 서로를 자동으로 인식하고 연결되는 상태로 만들어지는 것 아닌가?

답변 이미지 자체에는 서로에 대한 정보가 전혀 담겨 있지 않습니다. 연결 가능한 상태가 되는 것은 이미지가 빌드되는 시점이 아니라, docker compose up으로 컨테이너가 실행되는 시점에 별도로 마련되는 두 가지 장치 덕분입니다.

첫째, compose 파일의 networks 설정으로 모든 컨테이너가 같은 가상 네트워크(bridge)에 묶이며, 이 네트워크 안에서는 도커가 컨테이너 이름(서비스 이름)만으로 서로를 찾을 수 있는 내부 DNS를 자동으로 제공합니다. 둘째, environment 항목에 http://eureka-server:8761처럼 다른 서비스의 이름을 직접 명시해 두어, 애플리케이션이 그 이름으로 접속을 시도하도록 알려줍니다. 이 두 설정이 없다면 이미지는 8개가 똑같이 만들어지더라도, 컨테이너들은 서로를 전혀 찾지 못한 채 각자 고립되어 실행됩니다.

```
auth-server:
  environment:
    - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
  networks:
    lecture-net:
      driver: bridge
```

질문 5 --add-host host.docker.internal:host-gateway 설정을 docker-compose.yml에서는 어떤 항목으로 대체할 수 있는가?

답변 compose 파일에서는 extra_hosts 항목으로 동일한 설정을 표현합니다. docker run의 --add-host 옵션과 정확히 같은 역할을 하며, 서비스 정의 안에 한 줄만 추가하면 됩니다.

```
services:
  app:
    extra_hosts:
```

- "host.docker.internal:host-gateway"

IX. 컨테이너 내부 진입과 실습

컨테이너가 정상적으로 떠 있는지, 네트워크가 제대로 뚫려 있는지 직접 확인하고 싶을 때는 실행 중인 컨테이너 내부로 들어가 진단할 수 있습니다. 이 장에서는 그 방법과 흔히 마주치는 문제를 다룹니다.

질문 1 이미 실행 중인 컨테이너 내부로 들어가 이것저것 직접 테스트해 보려면 어떤 방법을 사용해야 하는가?

답변 `docker exec -it 컨테이너이름 /bin/sh`(또는 `bash`)로 접속할 수 있습니다. 베이스 이미지가 `eclipse-temurin:21-jre`처럼 경량 이미지라면 `bash`가 없을 수 있으므로 이 경우 `sh`로 접속해야 합니다.

접속 후에는 `env`로 환경변수가 잘 들어왔는지, `ps aux`로 프로세스가 정상 기동했는지, `curl`로 애플리케이션이 실제로 응답하는지 등을 직접 확인할 수 있습니다.

```
docker exec -it loginauth-extended-app /bin/sh
env | grep DB_
ps aux
```

질문 2 컨테이너 내부에서 `nc`(넷캣) 명령으로 네트워크 연결을 확인하려 했으나 `command not found` 오류가 발생하는 경우, 어떻게 대처할 수 있는가?

답변 JRE 기반 최소 이미지에는 `nc`, `curl`, `ping` 같은 진단 도구가 아예 포함되어 있지 않은 경우가 많습니다. 이때는 세 가지 방법을 시도할 수 있습니다. 첫째, `bash`가 있다면 별도 설치 없이 `cat < /dev/tcp/호스트/포트` 구문으로 포트가 열려 있는지 확인할 수 있습니다.

둘째, `docker exec -u root`로 `root` 권한으로 접속해 필요한 도구를 임시로 설치할 수 있습니다. 셋째, 가장 깔끔한 방법으로 대상 컨테이너와 네트워크만 공유하는 별도의 진단 전용 컨테이너(예: `nicolaka/netshoot`)를 붙여서 그 안에서 `nc`, `dig` 같은 도구로 확인하는 방법이 있습니다.

```
# 방법 1: bash의 /dev/tcp 트릭
cat < /dev/tcp/host.docker.internal/53301 && echo 포트 열림

# 방법 3: 진단 전용 컨테이너 사용
docker run -it --rm --network container:내컨테이너 nicolaka/netshoot
```

질문 3 `exit` 명령으로 컨테이너 접속에서 빠져나오는 행위는 어떻게 표현하는 것이 정확한가?

답변 "컨테이너 셸(셸) 세션에서 빠져나가기" 또는 "exec 세션 종료"라고 표현하는 것이 정확합니다.

중요한 점은 이것이 컨테이너 자체를 멈추는 것과는 다르다는 사실입니다. docker exec로 들어간 상태에서 exit하면 그 셸 세션만 종료될 뿐, 컨테이너는 백그라운드에서 계속 실행됩니다. 컨테이너 자체를 멈추려면 별도로 docker stop 명령을 사용해야 합니다.

정리 : 다시 개념 지도로

아홉 개 장을 모두 훑어보았다면, 다시 문서 맨 앞의 "학습 흐름 5단계"로 돌아가 전체 그림을 한 번 더 확인해 보시길 권장합니다. 개별 명령어와 옵션은 시간이 지나면 잊어버릴 수 있지만, "컨테이너는 격리되어 있고, 우리가 배운 도구들은 그 격리에 필요한 만큼만 구멍을 뚫는 장치"라는 하나의 원리를 이해하고 있다면, 새로운 옵션이나 상황을 마주했을 때도 스스로 추론해서 답을 찾아낼 수 있습니다.

학습 순서 제안

- I 장(이미지 개념)과 V 장(포트와 네트워크)을 가장 먼저 확실히 이해합니다. 이 두 장이 "격리"라는 뿌리 개념을 가장 직접적으로 보여줍니다.
- 그 다음 II 장과 III 장(Dockerfile 문법과 멀티스테이지 빌드)을 통해, 이미지가 실제로 어떻게 조립되는지 손으로 직접 따라 만들어 봅니다.
- IV 장과 VI 장, VII 장(실행·빌드 옵션, 환경변수)은 필요할 때마다 사전처럼 찾아보아도 충분합니다.
- 여러 컨테이너를 함께 운영해야 하는 시점이 오면 VIII 장(Docker Compose)으로 넘어가고, 문제가 발생했을 때는 IX 장(컨테이너 내부 진입)의 방법으로 직접 진단해 보는 습관을 들입니다.

참고 : 이 문서는 실제 Spring Boot 도메인 확장 최종 프로젝트 (loginauth-extended : Dockerfile, docker-compose.yml)를 기준으로 작성되었으며, 초보자에게 발생 가능한 질의 응답을 기반으로 재구성한 학습 자료입니다. 또한 MSA 개발에서 멀티 컨테이너 실습 배포 구조에 대한 설명을 포함합니다. 실습 환경에 따라 세부 동작이 다를 수 있으므로, 직접 명령어를 실행해 보며 확인하는 과정을 병행하시길 권장합니다.